Q1. Load the TCS_history.csv

Q2. Do the some descriptive analysis

Q3. Apply Linear Regression, polynomial, and RBF for each model find the confidence level:

confidence level= right predicted values/test size

Q4. Choose that model which have highest confidence level.

Q5.Predict the close price with random dates as input

Date	Close price.

Assignment 1.1

Q1. Load energy dataset.

Q2. Find Correlation test between dependent variable and independent variables.

Use H0 : $\rho = 0$ (There is no correlation between two variables)

H1 : $\rho \neq 0$ (There is correlation between two variables)

Output

Variable Correlation coefficient p-value

Justify answer.

Q3. Find the Coefficients , T.Values and P.Values.

Q4. Find the linear regression equation using function as well as formula given below

B =

Q5. Apply the Baysian Linear Regression and find the following values in Table

Variables	Coeff.	Quantile(2.5%)	Qunatile(97.5%)
Slope			
X1			
X2			

.

Q6. Find the linear regression equation and plot it with linear regression plot .

Q7.Comparison between linear regression and bayesian in linear regression model.

Method	RMSE	MAD	MAPE
LR			
Baysian			

MAD (Mean Absolute Deviance) and MAPE (Mean Absolute Percentage Error)

Conclude your Results.

Assignment 2.1

Q1. Load USA housing dataset.

Q2. Find Features of Data-set and Number and Types .

Q3.plot the USA housing histogram for price.

Q4.plot Heat map of the correlation between each of columns.

Q5. Plot Histograms and correlation scatter plots.

Q6. Is Data visualization and Fit for Linear Regression? Check.(Use Correlation of each feature with y)

Q7. Divide the dataset as 40% and 60% into test data and training data, respectively.

Q8.Apply Linear Regression on Training Dataset and Calculate Coefficients.

Q9. Apply Predict() Method to calculate Predicted value for Test Dataset.

Q10. Calculate Error matrix MAE, MSE, and RMSE

Q11. Using distplot to plot predicted values and test values for correctness.(Scatter plot: Y Test versus Prediction)

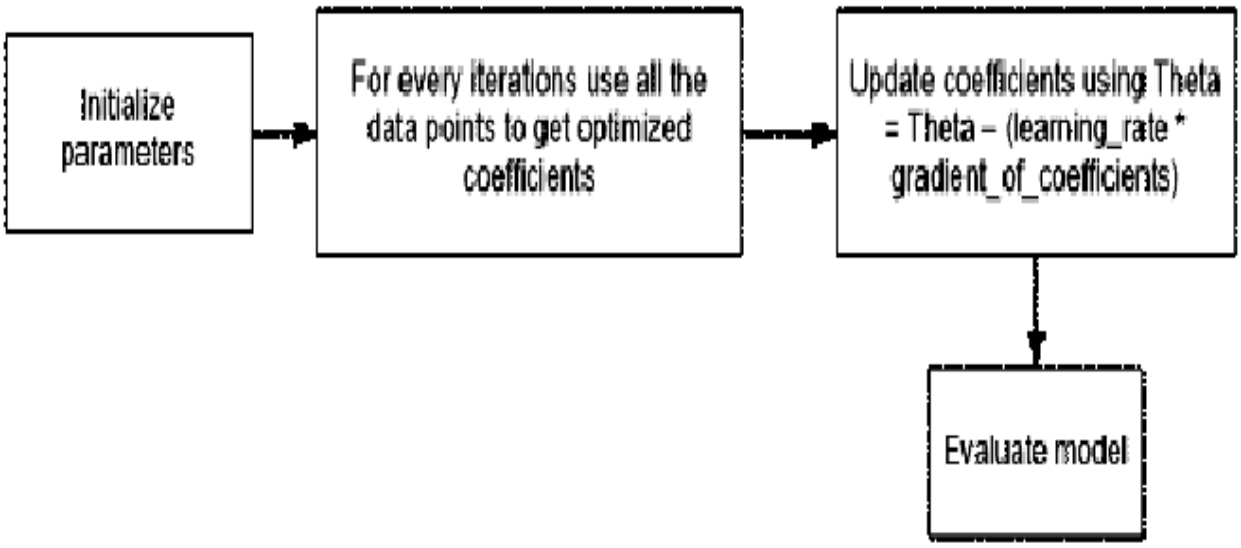
Q12. Plot the Residual Histogram.

Supplement:

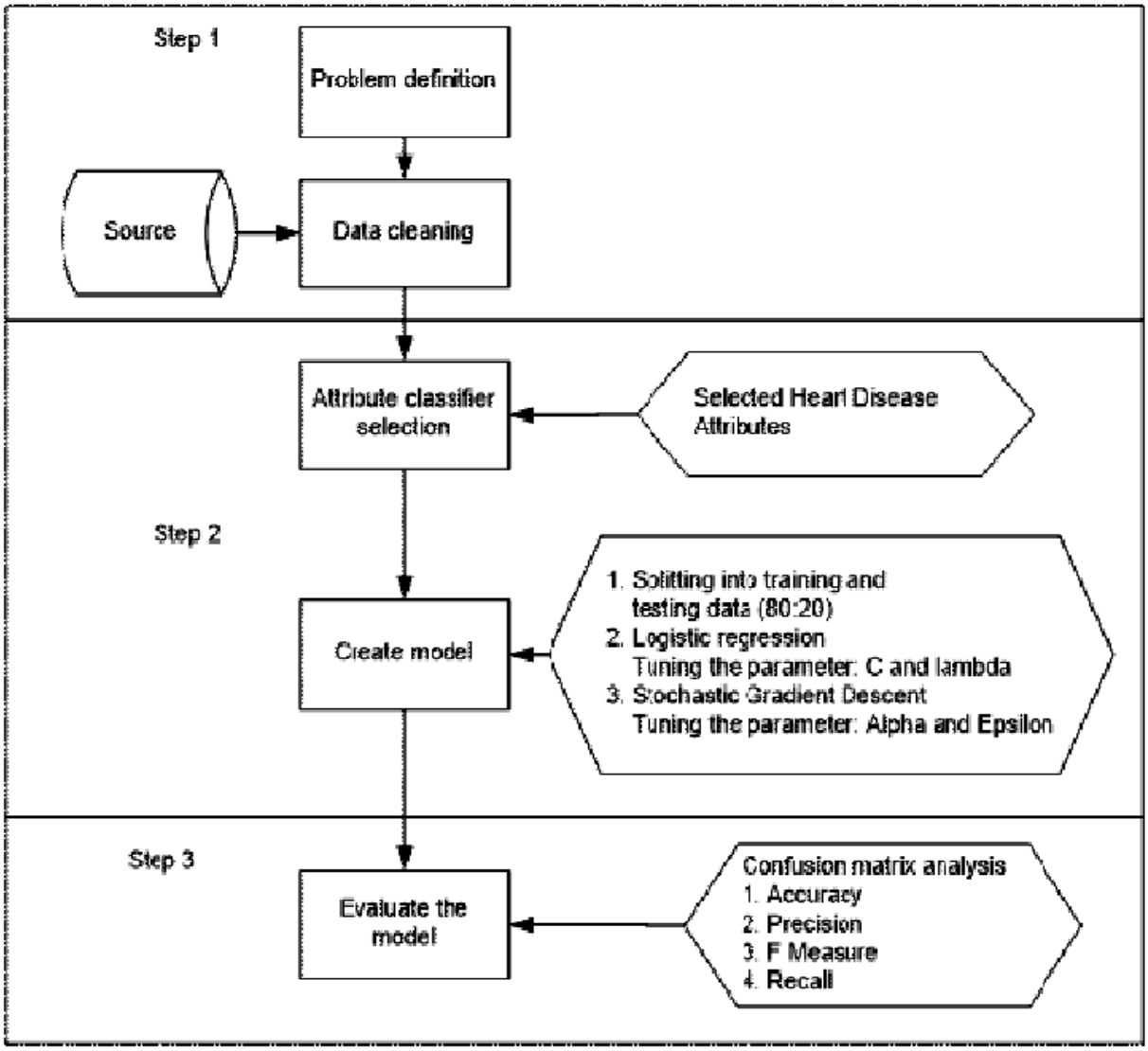
MAE =

MSE =

RMSE =


Initialize parameters
For every iterations use all the data points to get optimized coefficients
Update coefficients using Theta = Theta – (learning_rate * gradient_of_coefficients)
Evaluate model

SGD



Flowchart of Experiment

Q1. Select type and descriptions of dataset.

Q2. Take Logistic regression in the phyton library. Parameter tuning for optimizing classifier performance set for several parameter values namely: C =1.0, λ = 1.2 C .The default value is 1.0. The smaller C value specifies the stronger regularization value. The Lambda parameter used to control the regularization strength. The increase in the lambda values the strength of the regularization effect.

Q3. Find confusion matrix using logistic regression without SGD.

Q4. Find ACCURACY, PRECISION, F-MEASURE, RECALL: LOGISTIC REGRESSION CLASSIFIER

Q5. Learning rate that is derived from alpha and epsilon value. The learning rate managed the number of modification for estimating error. The alpha parameter is a constant value that multiplies the regularization term. The higher the value, the stronger the regularization. The default value is 0.0001. Epsilon parameter controlled the threshold value for getting precise prediction. Any distinction between the current prediction and the correct value is unobserved when this value less than the threshold value. The default value is 0.1. Experiment on this study set alpha value of 0.0001 and epsilon value of 0.1.

Q6. . Find confusion matrix using logistic regression with SGD.

Q7. . Find ACCURACY, PRECISION, F-MEASURE, RECALL: LOGISTIC REGRESSION CLASSIFIER with SGD.

Q8. Plot a graph Classifier result: Logistic regression and with Stochastic Gradient Descent for ACCURACY, PRECISION, F-MEASURE, RECALL.

Assignment 4

Q1. Load load_breast_cancer() built in dataset.

Q2. convert dataset to pandas dataframe.

Q3. Append dataframe containing tumor features with diagnostic outcomes.

This labels will be used for supervised learning.

Q4.Find relation in diagnosis by using mean.

Q5. create to dataframes - one for positive, one for negative(diagnosis).

Q6. Visualize tumor characteristics for positive and negatives diagnoses

Q7. Create 'for loop' to enerate though tumor features and compare with

histograms(20:-1)

Q8. Find Visualization of relationships between features and diagnoses.

Q9. Split data into testing and training set. Use 80% for training

Q10. normalize features to account for feature scaling

Q11. Apply DT, NB, KNN Logistic Regression and SVm to find accuracy, F1-Score and Recall.

Q12. Further apply the PCA to do the same exercise in Q11.

Q13. Do the Comprative study with PCA and WITHout PCA.

Question 1.1

Q1. Apply Mc-Culloch Pits model for the following Boolean operations:

Plot the decision boundaries for 2 - dimensional and 3-dimensional.

Implement AND with 3-variables and threshold 3.

Implement OR with 3-variables and threshold 1.

Implement X 1and !X2 with 2-variables and threshold 1.

Implement NOR with 2-variables and threshold 0.

Implement NOT with 2-variables and threshold 0.

Implement OR with 2-variables and threshold 1.

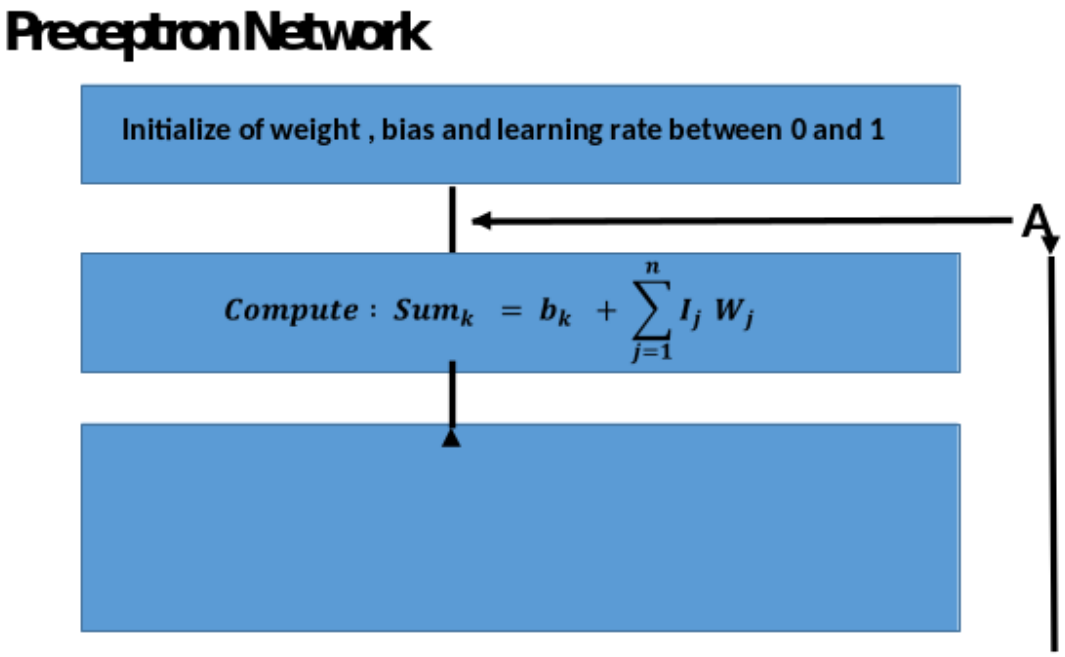
Implement AND with 2-variables and threshold 2.

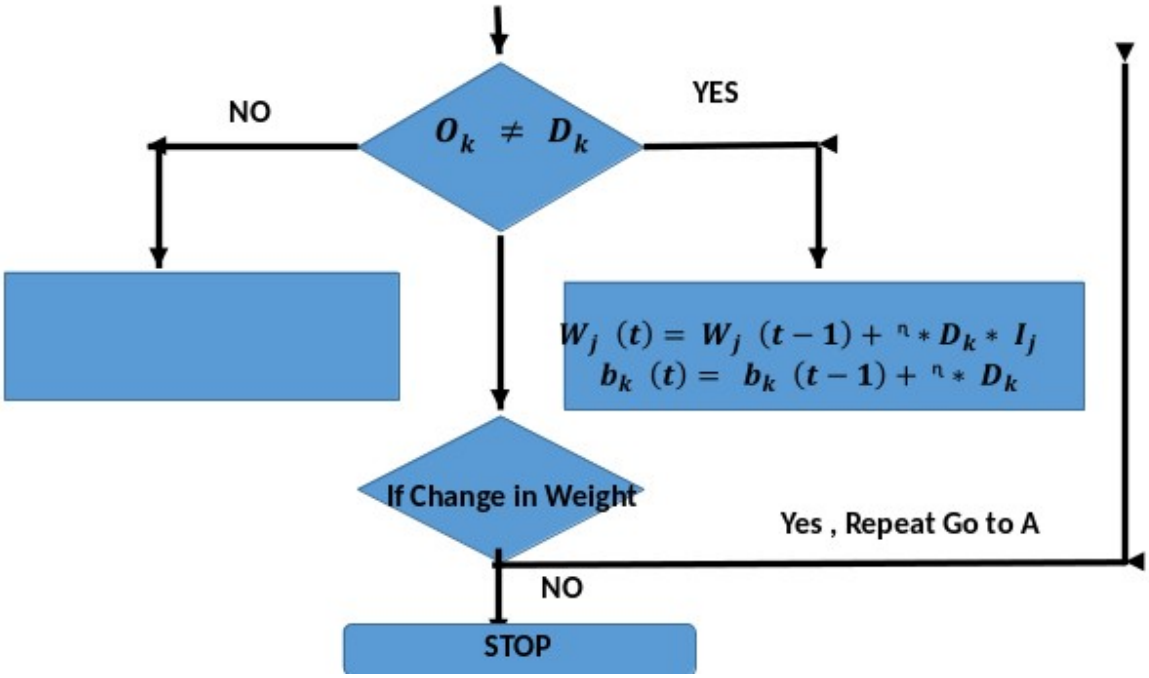
Implement OR with 3-variables and threshold 1.

Assignment 5

Q1. Implement the OR and AND problem using perceptron learning model.

Algo





- transfer function used in the output neuron of the perceptron

- $f(Sum_k\ ) = \begin{cases} 1\ if\ Sum_k\ > \theta_k \\ 0\ if - \theta_k \leq Sum_k\ \leq \theta_k \\ -1\ if\ Sum_k\ < -\theta_k \end{cases}$

- θ_k =0.5 , W1=0, W2=1, n=1
- Then the weights will be updated

Implement AND and OR.

In each iteration outputs updated weights and t-y and output.

Q2. Implement same with vector notation.

Where $y=g(W^TX)$ and $W^T=W^T(old)+\alpha(t-y).X^T$

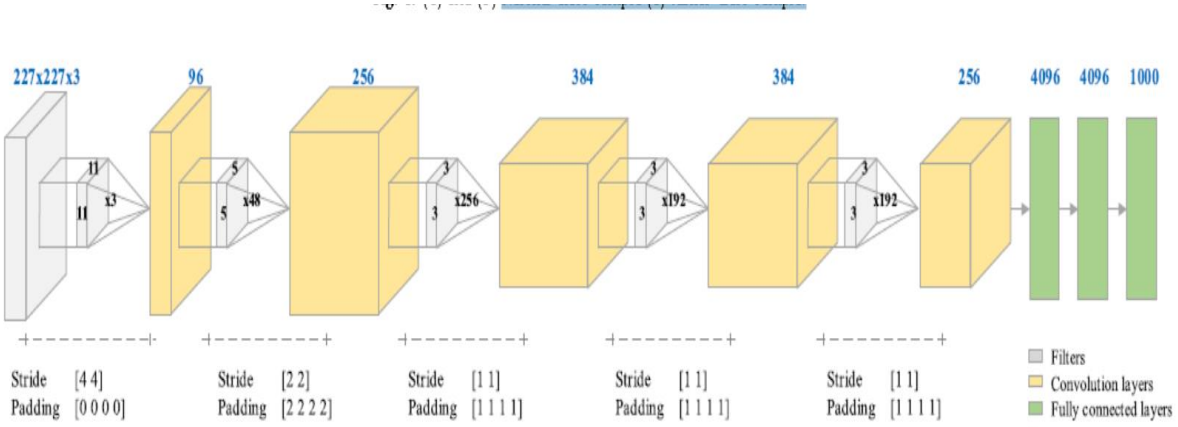
Q3. Download the Fashion MNIST dataset from https://github.com/zalandoresearch/fashion-mnist and develop an image classification

Lab Assignment 6

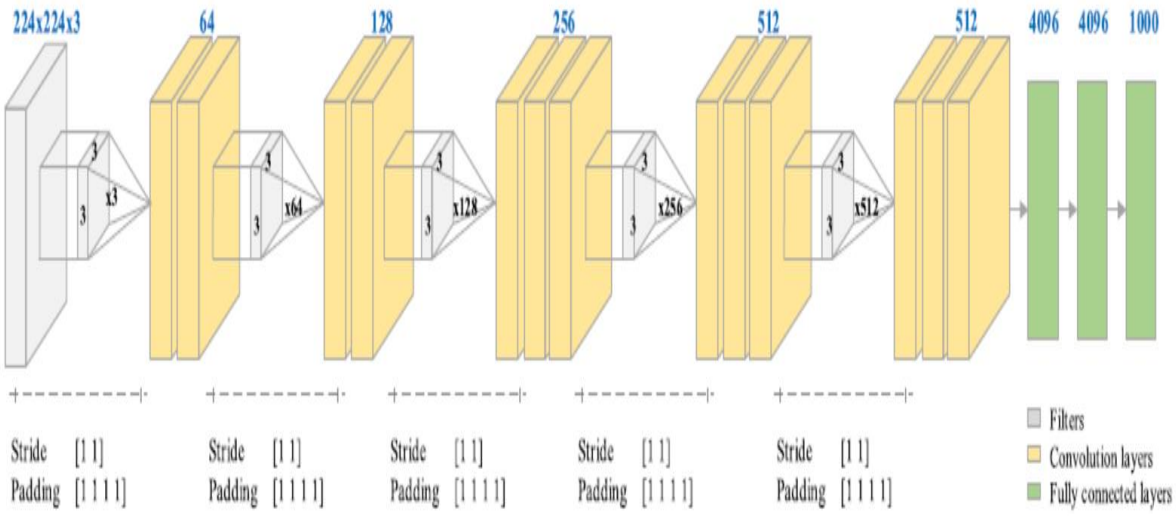
Classification of brain MRI using hyper column technique with convolutional neural network and feature selection method

1. The dataset consists of open-access brain tumor MRI containing two classes of the tumor and normal
   The dataset consists of 155 and 98 tumor and normal brain MRI, respectively. The dataset is heterogeneous MR images collected from 253 patients.
   The images in the dataset has not high resolution and each image has a different resolution.
   The image format is JPEG.

2. Print the Normal class sample and tumor class sample.



(a) AlexNet



(b) VGG-16

The AlexNet is one of the popular CNN architectures that famed its name in the ImageNet competition in 2012
This architecture consists of convolutional, pooling and fully connected layers.
The input size is 227 × 227 pixels. It is based on the process of moving 3 × 3 or 5 × 5 pixel filters on the image in the convolution layer.
Thus, activation maps containing more efficient features are created for transfer to the next layer.
The most important feature of activation maps is that they have unique features The pooling layer is used to reduce the size and cost of the image without disturbing its features

VGG-16 architecture consists of convolutional, pooling and FC layers like AlexNet architecture. It contains a total of 21 layers
This architecture has an increasing network structure.
The input size is 224 × 224 pixels. The filter size in the convolutional layer is 3 × 3 pixels. In this architecture, the final layers have a fully connected layer used for feature extraction.

In this study, CNN architectures are used for feature extraction. 10 0 0 deep features were extracted from each FC-8 layer of the models . For both models, the filter size is set to 3 × 3 pixels, the number of strides is two, and the docking type is maximum.
In addition, the dataset was divided into two classes as training and test sets with 70% and 30% rates.

Image augmentation technique

Data augmentation techniques are useful to balance the distribution of the samples over the classes when imbalanced datasets are available

In this study, the image augmentation technique was used to equalize the number of samples in the normal class brain tumor MRI to the number of brain tumor MRI class. For this purpose, the image augmentation methods provided by the Keras library in Python were used (Francois Chollet, 2016). For normal-class MRI, rotation, width, and height changing, cutting, zooming, horizontal rotating brightening and filling operations were performed.

The degree of image rotation was set to randomly generate from 0 to 20. The values of the width-height changing, zooming, and the horizontal rotating ratio parameters were adjusted to 0.15. Two samples belonging to the normal-class brain tumor MRI are shown in Fig. 4 .

A subset of the images obtained by using the data augmentation technique is presented in Fig. 5 .

Print the normal brain tumor and normal classes images after data augmentation.

Hypercolumn technique

CNN architectures use the features of the latest layer in their architecture to perform classification or segmentation. The features in the previous layers are not moved to the last layer. The hypercolumn technique allows the inherent characteristics of the previous layers to be transferred to the last layer in a vector In other words, the hypercolumn in a pixel is a vector of activations of all CNN units above that pixel. Thus, the spatial position information can be retrieved from the previous layers and a more accurate prediction result can be obtained. This enables the network to achieve better results . Fig. 6 shows an example of applying the hypercolumn technique on the original image. When Fig. 6 is examined, the bottom image is the input image and the processing steps above it represent the feature map in the layers of the CNN model. A red icon indicates a pixel. Each pixel in the last layer of the CNN model carries the features of the previous layers vectorically. The hypercolumn technique is actually a masking technique. This technique stacks the efficient features obtained from the previous layers of the CNN model on the original image. This makes the original image more visible and increases the number of features.

A subset of the mask images obtained from the images by the hypercolumn technique is shown in Fig. 7 . Hypercolumn masks carry the characteristics of layers in the CNN architecture of each pixel of the original image as a series of vectors. In this study, colored masks were obtained because three main color channels were used in the masking step. The goal is to produce more efficient features.

Because color channels affect the depth and resolution of the input image

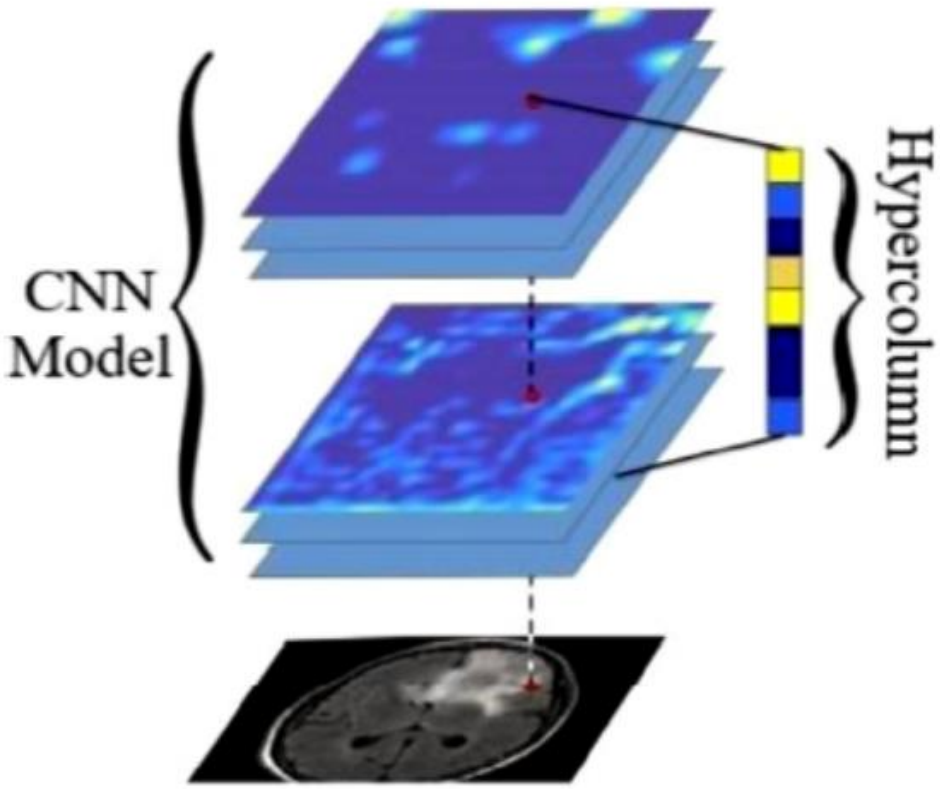


Fig. 6. Hypercolumn illustration.

Feature selection method

The RFE is known as a recursive feature reduction method. This method handles all the features obtained in the dataset and evaluates them in order of their priority. If the variable R is considered to be an array representing the features, multiple features sequences are sorted by priority R1 > R2 > R3. In this method, each time an iteration is performed, the top-ranked features of the Ri sequence (the most efficient sequence of features) are maintained, improved, and accuracy is reassessed. In the next step, the best accuracy values are compared with the Ri sequence, and if the accuracy values are better than the most efficient Ri features, the accuracy values are updated in due to importance in the model (Sean Escanilla et al., 2018). In this study, the RFE feature selection method is implemented by using the open-source code in Python

Table 1
Parameter values of CNN models.

Software used	CNN Architecture	Image Size	Optimization	Momentum	Decay	Beta	Mini Batch	Learning Rate
Matlab	AlexNet VGG-16	227 × 227 224 × 224	Sigmoid Gradient Descent	0.95	1e-6	–	16	0.0001

Proposed method

The proposed method consists of three basic steps. In the first step, image augmentation techniques are used to ensure that the data is distributed equally between the classes. Thus, more efficient features can be extracted by CNN models. In the second step, it is aimed to make the images more prominent (creating new efficient features) by using the hypercolumn masking technique on the balanced dataset.

This is done on the image pixels in the FC-8 layer of both models. In the used CNN models, the features obtained from the previous layers are kept in the FC-8 layer in a vector format.

In the last stage, a total of 20 0 0 deep features obtained from the FC-8 layers are evaluated by the RFE feature selection method, and the dimension of the feature set is reduced. In the last step, the SVM classifier is employed. In the feature selection method, the SVM classifier is chosen for its generalization performance on realworld problems. The overall design of the proposed method is shown in Fig. 8 . Other parameter selections of the CNN models used in this study are used as shown in Table 1 . Mini-batch value is the case where more than one input is processed on the model at the same time. This may interfere with the operation of the model if the computer's hardware resources are not sufficient. Therefore, the mini-batch value was chosen as 16 in this study.

Experimental results

In this study, the experiments were divided into three steps. In the first step, the balanced dataset was directly trained by using only CNN models without using the hypercolumn and feature selection method. SVM was used as a classifier in this experiment. The success rate was 90.32% with AlexNet and 87.10% with VGG-16. In the second step, two CNN architectures were processed with the hypercolumn technique and classified with the SVM model. In this experiment, it was aimed to observe the contribution of the hypercolumn technique. The achieved success rate was 92.47% with AlexNet and 90.32% with VGG-16. The contribution of the hypercolumn masking technique to the classification process was 2.38% in AlexNet and 3.68% in VGG-16.

In the third step, the features of CNN models under hypercolumn masking were combined (10 0 0 + 10 0 0 = 20 0 0). Then, the RFE feature selection algorithm was applied on 20 0 0 deep features and sub-feature sets of 10 0, 20 0, 30 0 and 400 features were obtained respectively. In this step, it was aimed to increase the classification success with the selected features by using the RFE. As a result, the best performance was achieved with an accuracy of 96.77% by using 200 selected deep features. In this study, all results of the experiments are shown in Table 2 . In addition, when Table 2 is examined, it can be seen that selecting 400 features was not used because of decreasing classification accuracy.

As a result, the combined dataset achieved the best performance with 200 features with the hypercolumn masking technique and the RFE feature selection method. The confusion matrix and ROC curve of this analysis are shown in Fig. 9 . The contribution of the proposed model on the classification performance is shown in Fig. 10 .

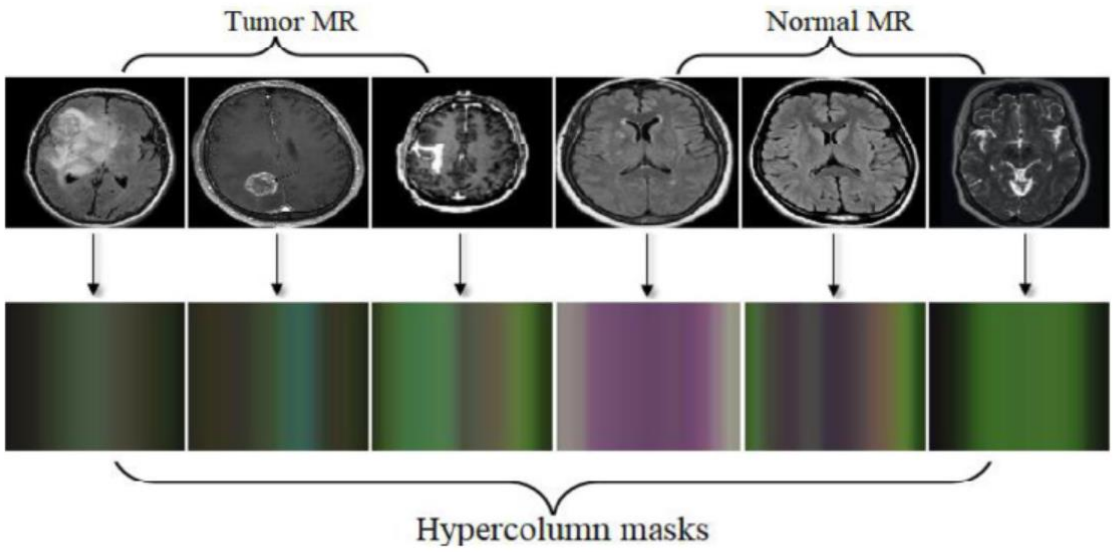


Fig. 7. A subset of the hypercolumn masks created with the hypercolumn technique of tumor and normal data.

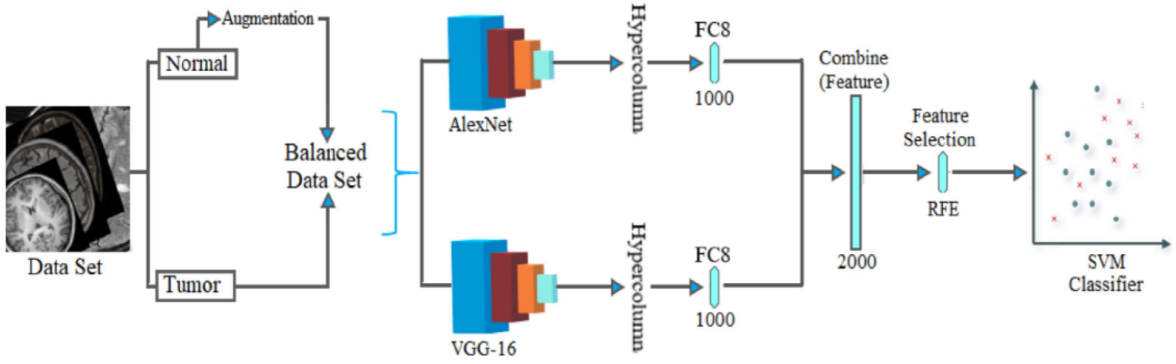


Fig. 8. The overall block diagram of the proposed model.

Find out the following results

Table 2
Classification results of the experiments performed in this study.

CNN Architecture	Hypercolumn Technique	Features (FC8)	RFE / Features	Sensitivity (%)	Specificity (%)	F-Score (%)	Accuracy (%)
AlexNet	No	1000	No / -	95.24	86.27	89.89	90.32
VGG-16	No	1000	No / -	87.23	86.96	87.23	87.10
AlexNet	Yes	1000	No / -	95.45	89.80	92.31	92.47
VGG-16	Yes	1000	No / -	89.58	91.11	90.53	90.32
AlexNet & VGG-16	Yes	2000	Yes / 100	95.65	95.74	95.65	95.70
AlexNet & VGG-16	**Yes**	2000	**Yes / 200**	97.83	95.74	96.77	**96.77**
AlexNet & VGG-16	**Yes**	2000	**Yes / 300**	95.83	97.78	96.84	**96.77**
AlexNet & VGG-16	Yes	2000	Yes / 400	95.74	95.65	95.74	95.70

Appendix

Hypercolumns are a concept in Convolutional Neural Networks (CNNs) that address the need to capture rich feature representations from multiple layers of the network. The idea was introduced in the paper "Hypercolumns for Object Segmentation and Detection" by B. Hariharan, P. Arbeláez, R. Girshick, and D. Malik, presented at CVPR 2015.

What is a Hypercolumn?

A hypercolumn is essentially a vector of features extracted from multiple layers of a CNN for a specific spatial location. Instead of relying on features from a single layer, a hypercolumn aggregates features from various layers to capture a more comprehensive representation of that location. This approach can be particularly useful for tasks like object detection, segmentation, and any other task requiring detailed spatial information.

Key Concepts

1. **Feature Aggregation**:
    - **Layer Aggregation**: In a traditional CNN, features are extracted from specific layers. A hypercolumn collects features from multiple layers (e.g., convolutional layers, pooling layers) at the same spatial location.
    - **Rich Representation**: By combining features from different depths, the hypercolumn provides a richer and more detailed representation of that point in the image, capturing both

To implement hypercolumns in a CNN using Python, you will typically use a deep learning framework like TensorFlow/Keras or PyTorch. Here's a step-by-step guide on how to create a hypercolumn representation using TensorFlow/Keras:

Step-by-Step Implementation

1. Set Up Your Environment

Make sure you have TensorFlow and Keras installed. You can install TensorFlow with:

```
bash
Copy code
pip install tensorflow
```

2. Define Your CNN Model

You first need a CNN model from which to extract hypercolumns. Here's a simple example of a CNN model:

```
python
Copy code
import tensorflow as tf
from tensorflow.keras import layers, models

def create_cnn_model(input_shape):
    inputs = tf.keras.Input(shape=input_shape)

    # Define the CNN layers
    x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(inputs)
    x = layers.MaxPooling2D((2, 2))(x)
    x = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(x)
    x = layers.MaxPooling2D((2, 2))(x)
    x = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(x)
    x = layers.MaxPooling2D((2, 2))(x)

    # Define the model
    model = models.Model(inputs=inputs, outputs=x)
    return model
```

3. Extract Features from Multiple Layers

To create a hypercolumn, you need to extract features from multiple layers of the CNN model. You can do this using Keras' functional API.

```
python
Copy code
def extract_features(model, layer_names, input_image):
    features = []
    for layer_name in layer_names:
        layer_output = model.get_layer(layer_name).output
        feature_model = models.Model(inputs=model.input, outputs=layer_output)
        features.append(feature_model.predict(input_image))
    return features
```

4. Concatenate Features to Form Hypercolumns

Once you have extracted features from the desired layers, you concatenate them to form the hypercolumn representation.

```
python
Copy code
import numpy as np

def create_hypercolumn(features):
    # Concatenate features along the channel axis
    hypercolumn = np.concatenate(features, axis=-1)
    return hypercolumn
```

5. Putting It All Together

Here's how you can use the functions defined above to create a hypercolumn representation:

```
python
Copy code
# Define input shape and create model
input_shape = (128, 128, 3)  # Example input shape
cnn_model = create_cnn_model(input_shape)

# Assume you have some input image
input_image = np.random.rand(1, 128, 128, 3)  # Example input image

# Extract features from specific layers
layer_names = ['conv2d', 'conv2d_1', 'conv2d_2']  # Names of layers you want to
extract features from
features = extract_features(cnn_model, layer_names, input_image)

# Create hypercolumn representation
hypercolumn = create_hypercolumn(features)
print('Hypercolumn shape:', hypercolumn.shape)
```

Summary

- **Create a CNN model**: Define a CNN architecture with multiple convolutional layers.
- **Extract features**: Get feature maps from various layers of the model.
- **Concatenate features**: Combine features from different layers to form the hypercolumn representation.

This example uses TensorFlow/Keras to demonstrate the concept of hypercolumns. The key steps involve defining a CNN, extracting features from multiple layers, and then concatenating these features to create the hypercolumn. This approach allows you to leverage rich multi-level feature representations for tasks like segmentation or object detection.